

MemSens: Significantly Reducing Memory Overhead in Adjoint Sensitivity Analysis Using Novel Error-Bounded Lossy Compression

Chenxi Li*, Yihang Feng*, Fuxing Deng*, Dingwen Tao[†], Weifeng Liu* and Zhou Jin[‡]

*SSSLab, Dept. of CST, China University of Petroleum-Beijing, Beijing, China

[†] Institute of Computing Technology, Chinese Academy of Sciences, Beijing, China

[‡]College of Integrated Circuits, Zhejiang University, Hangzhou, China

Email: z.jin@zju.edu.cn

Abstract—Adjoint sensitivity analysis is an exceptionally efficient method for computing the gradient of an objective function with respect to given parameters, playing a crucial role in modern circuit design and verification. According to the principles of the adjoint method, it is necessary to store all essential system state information, such as state vectors and Jacobian matrices, at each time step during the forward integration process in order to construct the adjoint equations during the backward integration. Therefore, the memory overhead of the adjoint method is proportional to the system size and the number of time steps, resulting in prohibitive memory costs for solving large-scale dynamic systems.

In this paper, we propose a novel, memory-efficient adjoint sensitivity analysis method that significantly reduces the memory overhead of storing system state information by employing error-bounded lossy compression techniques. Our compression algorithm effectively utilizes the spatiotemporal characteristics of data in circuit simulations and incorporates stringent error control mechanisms. This approach achieves a two-order-of-magnitude reduction in memory overhead during simulation while ensuring that the accuracy of the adjoint solution remains unaffected.

Index Terms—Lossy compression, Circuit simulation, Adjoint sensitivity analysis

I. INTRODUCTION

Sensitivity analysis involves calculating the gradient of the objective function with respect to given parameters, serving as a powerful tool for evaluating how perturbations in system parameters impact system outputs. This method provides significant guidance in modern integrated circuit (IC) design and verification, making it widely applied in areas such as performance modeling [1], circuit optimization [2, 3], and yield estimation [4]. Sensitivity analysis encompasses two methods: the direct method and the adjoint method. The direct method is not suitable for dynamic systems with a large number of parameters [5], hence the adjoint method has become the standard in modern simulations.

A major drawback of the adjoint method is its high memory overhead [6]. During the forward solution process, it is necessary to store critical historical information at each time step, such as the state vectors, to construct differential equations during the adjoint process [7]. In large-scale simulations, this

approach incurs substantial spatial and temporal costs. An alternative strategy involves storing additional state variables, such as the Jacobian matrix at each time step, to avoid recomputation during the reverse process and thereby reduce computational time. However, this approach further exacerbates the issue of memory overhead [8]. It is worth noting that during the simulation, Jacobian matrices can share indices [9], meaning that the primary memory overhead in adjoint sensitivity analysis stems from storing large-scale floating-point data.

One potential approach to reducing memory overhead during simulation is data compression. However, the mantissa of floating-point numbers often exhibits significant randomness, which limits the compression ratio achievable with traditional lossless compression algorithms [10], rendering them insufficient for the demands of large-scale simulations. Fortunately, advanced error-bounded lossy compression algorithms offer a promising solution to this challenge.

Error-bounded lossy compression, a current research hotspot in the field of high-performance computing (HPC), can significantly reduce data size while ensuring accuracy. It is widely regarded as the best solution for addressing the challenges posed by large-scale scientific data [11]. However, in circuit simulations, the complexity of circuit spatial topologies and the diversity of device parameters result in data exhibiting significant spikiness in the spatial dimension. Most existing compression algorithms heavily rely on data smoothness [12, 13], making them ineffective for handling circuit simulation data.

To address the low compression ratio of existing algorithms on circuit simulation data, we propose a novel error-bounded compression algorithm that efficiently compresses vector and matrix data in circuit simulations while ensuring high precision. This algorithm leverages the unique characteristics of circuit-related vectors and matrices to effectively smooth simulation data. It then applies a hybrid predictor based on the properties of data regions to decorrelate the data. Finally, a rigorous data quantization mechanism and an efficient encoding scheme are used to significantly reduce the size of simulation data while preserving simulation accuracy.

To the best of our knowledge, this is the first error-bounded compression algorithm capable of efficiently compressing both matrices and vectors in circuit simulations. Experiments demonstrate that the proposed compression algorithm can reduce simulation memory overhead by two orders of magnitude.

II. BACKGROUND

A. Adjoint Sensitivity Analysis

In circuit design, sensitivity analysis is essential for evaluating the effects of parameter variations on performance. For sensitivity analysis, the circuit we are interested in can be represented by a set of differential algebraic equations (DAE) [14]:

$$\mathbf{g}(\mathbf{x}, t, \mathbf{p}) = \frac{d}{dt}\mathbf{q}(\mathbf{x}, \mathbf{p}) + \mathbf{f}(\mathbf{x}, \mathbf{p}) + \mathbf{b}(t, \mathbf{p}) = \mathbf{0} \quad (1)$$

where $\mathbf{x} \in \mathbb{R}^D$ is the state vector, t is time, $\mathbf{p} \in \mathbb{R}^M$ is the parameter vector with respect to which we require sensitivities, $\mathbf{q} \in \mathbb{R}^D$ and $\mathbf{f} \in \mathbb{R}^D$ represent the dynamic and static elements, respectively, $\mathbf{b} \in \mathbb{R}^D$ is the input. To obtain adjoint sensitivity, we take derivative of formula (1) with respect to $\mathbf{p}$:

$$\frac{d\mathbf{O}}{d\mathbf{p}} = \sum_{i=0}^n \sum_{k=i}^n \left(\frac{d\mathbf{O}}{d\mathbf{x}} \right)_k \left(\frac{\partial \mathbf{g}}{\partial \mathbf{x}} \right)_k^{-1} \left(\prod_{j=k-1}^i \left(\frac{1}{h_t} \frac{\partial \mathbf{q}}{\partial \mathbf{x}} \right)_j \left(\frac{\partial \mathbf{g}}{\partial \mathbf{x}} \right)_j^{-1} \right) \left(\frac{\partial \mathbf{q}}{\partial \mathbf{p}} \right)_i \quad (2)$$

This process is akin to the reverse of transient analysis, with the storage and reuse of matrices and vectors from the forward process significantly speeding up sensitivity analysis [15]. However, the requirement to preserve these matrices and vectors for numerical integration in the reverse process incurs substantial memory overhead. Thus, while sensitivity analysis is invaluable for design optimization, it also imposes significant computational resource demands.

B. Floating-Point Number

The IEEE 754 standard [16] defines floating-point (FP) data to be in 32-bit single-precision and 64-bit double-precision format. The single-precision includes one bit for the positive or negative sign, 8 bits for the exponent, and 23 bits for the mantissa. The double-precision format includes one sign bit, 11 exponent bits, and 52 mantissa bits, as demonstrated in Figure 1. The actual value of a floating-point data is represented as:

$$x = (-1)^{\text{sign}} \times 2^{(\text{exponent}-\text{bias})} \times 1.\text{mantissa}$$

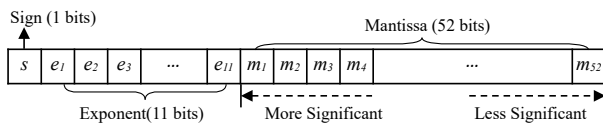


Fig. 1: IEEE 754 64-bit double-precision FP format.

C. Lossy Compression for Floating-Point Data

Error-bounded lossy compressors may have distinct designs and implementations. All existing error-bounded lossy compressors can be categorized into two groups: data-prediction based and domain-transform based.

Data-prediction-based compression model tries to predict each data point as accurately as possible based on its neighborhood in spatial or temporal dimension and then shrinks the data size by some coding algorithm, such as SZ [17] and FPZIP [18].

Domain-transform-based compression model needs to transform the original data to another non-orthogonal coefficient domain for decorrelation and then shrink the data size by applying some optimized coding algorithms such as embedded coding, e.g., ZFP [19].

However, all existing error-bounded lossy compressors are mainly designed to reduce storage overhead rather than memory overhead, considering error propagation. Only a few works [20–22] have utilized lossy techniques for memory compression, but these have not been optimized for the specific requirements of compression quality in circuit simulations.

III. MOTIVATION

A. Lossy Compression Brings Higher Compression Ratio

Traditional lossless compression algorithms are less effective for domain-specific data due to low compression ratios. In contrast, lossy compression algorithms can achieve higher compression by allowing controlled information loss. As shown in Table I, lossy methods offer compression ratios 1 to 2 orders of magnitude higher than lossless ones [23]. However, some lossy algorithms, such as ZFP and FPZIP, do not adapt well to circuit simulation data, resulting in relatively low compression ratios.

TABLE I: Compression ratio comparison.

Dataset	Lossless Compression			Lossy Compression under 1×10^{-2} Relative Error			
	GZIP	ZSTD	BZIP2	ZFP [19]	FPZIP [18]	SZ1.4 [17]	SZ2.1 [17]
add20(vec)	5.23	6.03	4.33	8.52	189.8	1824.81	1783.85
add20(mat)	2.52	9.74	7.19	14.6	52.51	509.418	214.24
mem_plus(vec)	2.57	2.08	3.45	7.34	67.27	1933.368	1515.33

B. Existing Lossy Compression Algorithms Introduce Significant Errors in Simulation

Circuit simulation, particularly during transient analysis, requires high precision. While lossy compression algorithms like SZ improve compression ratios (see Table I), they introduce significant errors that accumulate over time, compromising simulation accuracy, as shown in Figure 2. ZFP controls errors better, but its compression ratio remains low. Moreover, the varying error levels across different lossy compression methods underscore the challenge of maintaining data integrity. Therefore, there is an urgent need to develop lossy compression algorithms that reduce data size while preserving the precision necessary for reliable simulations.

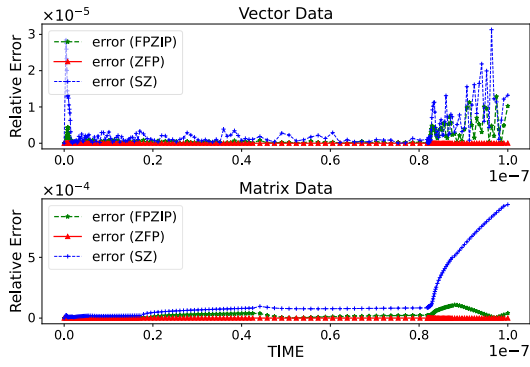


Fig. 2: The simulation error introduced by the compression algorithm under a relative error bound of 1×10^{-8} .

C. Existing Compressors Fail to Work Effectively on Circuit Simulation Data

As shown in Figure 3, the matrices and vectors generated by circuit simulation at each time step are spiky data with very low spatial autocorrelation. Existing compressors mostly rely on the smoothness of the data, so their performance will be greatly reduced on circuit simulation datasets. For example, ZFP, its domain transformation will lose effectiveness on simulation data, thus leading to low compression ratio. Due to the typically short time steps in numerical integration, data at adjacent time points exhibit strong correlations in the time dimension. This naturally suggests storing historical vectors or matrices as reference vectors or matrices to reduce data redundancy through prediction. However, this approach incurs significant memory overhead and struggles to handle dependencies in the data prediction process for lossy compression. Therefore, enhancing the smoothness of data in the spatial dimension is essential for simulation data compression.

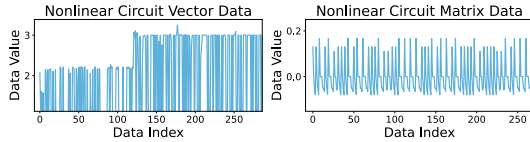


Fig. 3: The vector and matrix data of circuits exhibit pronounced spikiness.

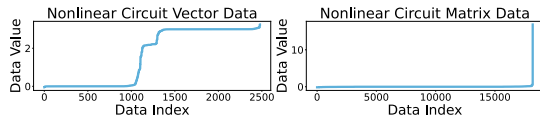


Fig. 4: After applying the reference sorting smoothing technique, the data exhibits strong autocorrelation.

IV. ERROR-BOUNDED LOSSY COMPRESSION FOR SIMULATION

The proposed error-bounded compression algorithm efficiently compresses vectors and matrices during simulation with minimal overhead while ensuring precise floating-point error control. It effectively resolves the memory overhead

challenges of traditional adjoint methods. As shown in Figure 5, the workflow consists of three main steps: (1) efficiently smooth the simulation data using **reference sorting**; (2) apply **RLE-FP** or **polynomial interpolation** predictors to decorrelate data based on regional characteristics; and (3) **quantize floating-point data** within a user specified error bound and encode it, achieving a significant data reduction.

A. Data Smoothing via Reference Sorting

Smoothing spiky data is essential for simulation data compression. Traditional smoothing techniques, such as wavelet transforms, are unsuitable due to their high time complexity [24]. To address this, we propose reference sorting, an efficient smoothing technique tailored for circuit simulation data. It consists of three main steps: (1) sort the floating-point queue in an ascending order at the initial time step, record the sorting indices, and use this sorting as a reference; (2) reorder the data in subsequent time steps based on the reference sorting; and (3) for longer integration processes, set validity check points to reinsert invalid data points into the sorted queue and update the reference sorting.

This approach is based on three key observations: (1) In circuits, nodes with large admittance (or small impedance) connections typically have identical or similar potentials [25]. Sorting the state vector groups these corresponding floating-point values together. This property is strongly linked to the circuit topology and device parameters, ensuring the reference sorting remains highly effective. (2) For matrices, elements with similar parameters contribute stamping values that are also similar [9], and sorting naturally clusters these corresponding floating-point values. (3) In time series data, values at adjacent time points are often similar or identical [26], leading to overall similarity in sorting across time steps.

Figure 4 illustrates the vector and matrix data of the nonlinear circuit at the 200th time step after applying the proposed smoothing technique. It can be observed that the data exhibits strong autocorrelation, which is highly advantageous for compression algorithms. The time overhead of reference sorting primarily arises from the sorting operation at the initial time step and the subsequent correction of a small number of out-of-order data points. The space overhead stems from storing the integer indices of a single time step, which is highly compressible and can be further reduced using integer compression algorithms. These time and space overheads are negligible compared to the original simulation costs.

B. Decorrelation by Hybrid Predictors

After data smoothing, we apply a hybrid predictor to decorrelate the data, significantly reducing its size. We observe that the smoothed data contains both numerically large steady regions and regions with small rapid numerical growth. To fully exploit the characteristics of these regional data patterns, we organize the entire dataset into different regions (or blocks) and apply distinct compression prediction techniques based on the data properties within each region.

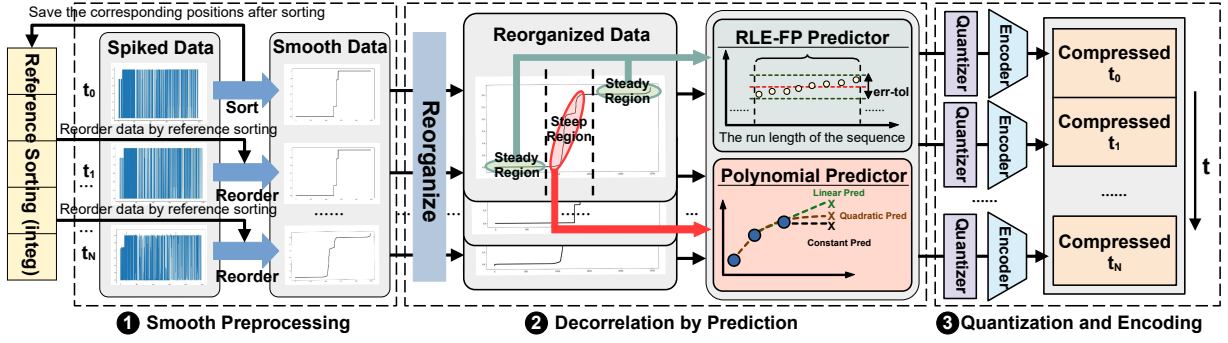


Fig. 5: The overall framework for error-bounded lossy compression of simulation data.

As shown in Figure 6, in the steady region, we introduce the RLE-FP technique for efficient decorrelation of the data. In the regions with rapid numerical growth, termed steep regions, we employ a prediction technique based on polynomial interpolation. This approach takes advantage of the varying data characteristics across different regions, optimizing the compression process.

We organize the sorted dataset into N regions. If the value at the right end of a region satisfies (3), we classify it as a steady region; otherwise, it is considered a steep region.

$$V_{right} < n \frac{V_{max} - V_{min}}{N} + V_{min} \quad (3)$$

In the text, V_{max} and V_{min} represent the maximum and minimum values in the dataset, respectively. n and N represent the total number of regions and the current region number from left to right, respectively. V_{left} represents the value at the leftmost end of the current region.

1) *RLE-FP*: The main idea of “RLE-FP” is to record consecutive values within the same error range in a floating-point sequence in order to achieve sequence compression.

Specifically, starting from the first floating-point number V_l at the left end of the steady region, we scan the floating-point sequence to identify the smallest $\hat{V}_m$ that satisfies $\hat{V}_m - V_l < e$ (where e is the user-specified error bound) and record this value as the center of the interval. From $\hat{V}_m$, we continue scanning to find the largest V_r such that $V_r - \hat{V}_m < e$, marking the right boundary of the interval. In other words, all floating-point numbers within this interval lie within a range centered at $\hat{V}_m$ with a radius of e . The process stops when a value outside the error range is encountered, and the length of the interval and $\hat{V}_m$ is recorded.

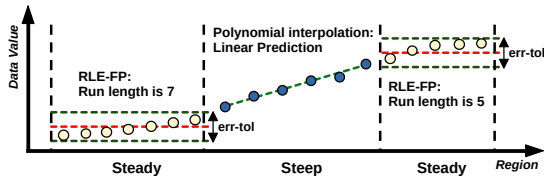


Fig. 6: Hybrid Predictors: RLE-FP for steady region and polynomial interpolation for steep region.

2) *Polynomial Interpolation Prediction*: The polynomial interpolation predictor is a predictor based on polynomial functions, which predicts future values by fitting historical data points. The current predicted value can be determined by the predicted values of the past n data points:

$$\hat{V}_n = \sum_{i=1}^n \alpha_{n-i} \hat{V}_{n-i} \quad (4)$$

To solve for the coefficient α , we substitute the historical predicted values into equation (4), yielding:

$$\hat{V}_n = \sum_{i=1}^n (-1)^{i+1} \binom{n}{i} \hat{V}_{n-i} \quad (5)$$

From equation (5), we can obtain the predicted value for each point, thereby achieving decorrelation in the steep regions. It is worth noting that using a polynomial interpolation predictor of order zero to two already provides sufficiently accurate predictions.

C. Error-Controlled Quantization and Data Encoding

A quantizer is responsible for quantizing the error between predicted values and true values, as well as handling unpredictable values. It significantly reduces the entropy of the original data while respecting the user-specified error boundaries.

We provide both absolute error bounds (denoted as ϵ) and relative error bounds (denoted as δ). To ensure simulation accuracy, both of these error bounds must be satisfied simultaneously. Therefore, there is a requirement between the true value V and the predicted value $\hat{V}$ (which is also the decompressed value):

$$|V - \hat{V}| \leq \epsilon \quad \text{and} \quad \frac{|V - \hat{V}|}{|V|} \leq \delta \quad (6)$$

So the final error bound e specified by the user is:

$$e = \min(\epsilon, \delta|V|) \quad (7)$$

Clearly, when $|V| < \frac{\epsilon}{\delta}$, we focus solely on relative error; in contrast, we pay more attention to absolute error.

The quantizer calculates the error between the predicted value and the true value. If the result falls within the error bound e , the quantizer represents the true value with the predicted value. Otherwise, we refer to this true value as unpredictable.

For unpredictable values, we perform an XOR operation with their neighboring unpredictable values, use 2 bits to record the number of leading zero bytes in the residual, and finally truncate the insignificant bit planes based on the user-specified error bounds.

By analyzing the binary representation of floating-point numbers, it can be concluded that to satisfy absolute error bounds, the minimum number of mantissa bits required is:

$$R_{abs} = \begin{cases} 0, & p(V) - p(\epsilon) < 0 \\ 52, & p(V) - p(\epsilon) > 52 \\ p(V) - p(\epsilon), & otherwise \end{cases} \quad (8)$$

Algorithm 1: Compressing Simulation Data at t_n .

Input: Floating-point array of a matrix or vector $\{V_1, V_2, \dots, V_M\}$ at t_n , user-specified error bound e
Output: compressed data stream in form of bytes

```

1: if  $t_n = start\_time$  then
2:   Sort the floating-point array.
3:   Save the sorting indices as reference sorting.
4: else
5:   if  $t_n = checking\_time$  then
6:     Check and maintain the validity of the reference sorting.
7:   end if
8:   Reorder the floating-point array by shared sorting.
9: end if
10: Organize data based on the regional characteristics of the sorted dataset.
11: for  $(V_i, i = 1, 2 \dots M)$  do
12:   if  $V_i$  is in a steady region. then
13:     if  $V_i$  is within the tolerance range. then
14:       ++run_length
15:     else
16:       Save run_length and predicted value.
17:     end if
18:   else if  $V_i$  is in a steep region. then
19:     Determine the predicted value  $\hat{V}_i$  using a polynomial predictor.
20:     Predict  $V_i$  using the  $\hat{V}_i$ .
21:   end if
22:   Quantize the unpredictable value and save the residual.
23: end for

```

Here, p represents the exponent of the floating-point number, and 52 denotes the number of mantissa bits for double-precision floating-points (23 for single-precision). To meet the absolute error bounds, larger floating-points require retaining more mantissa bits. Therefore, in implementations, the largest value within the range is used instead of V to determine the number of mantissa bits to retain. This approach not only improves precision but also ensures uniformity in the retained mantissa bits for all floating-points within the range.

To satisfy the relative error, the number of retained mantissa bits R_{rel} must satisfy $2^{R_{rel}} \leq \delta$. Therefore, we can derive:

$$R_{rel} = -p(\delta) \quad (9)$$

By calculating the significant bits to retain for floating-points using the above method and truncating the less important bit planes, we achieve quantization of unpredictable floating-points.

After quantization, we encode the indices generated during the compression process. Smaller integers can be represented with fewer bits, so we store only the differences between consecutive indices along with the first value. This reduces the magnitude of the integers, which are then encoded using variable-length encoding to significantly reduce the size of the indices. Finally, an advanced lossless compressor, ZSTD [27], is used for further compression of the data.

V. INTEGRATING COMPRESSION IN SENSITIVITY SIMULATION

As shown in Figure 7, we integrate the proposed error-bounded lossy compression algorithm into the simulation process, effectively reducing memory usage during both forward and backward propagation. In transient analysis, after successfully solving each time step, we store all the necessary state variables (e.g., state vectors and Jacobian matrices), denoted as D , and compress them to reduce storage. Subsequently, during the adjoint process, these variables are decompressed at the necessary moments to reconstruct the differential equations.

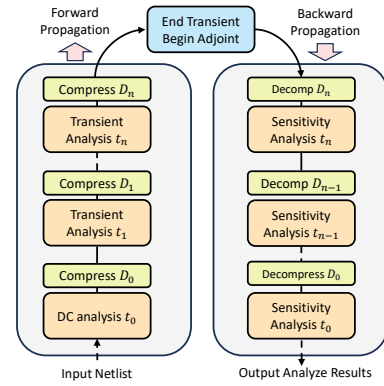


Fig. 7: Proposed memory-efficient simulation.

VI. EXPERIMENTAL EVALUATION

TABLE II: Dataset information.

Dataset	Circuit Type	#CirElem	Dimension Size	Data Size
ibm1t (vec)	RLC	76934	54265×1252	518.34MB
ibm1t (mat)	RLC	76934	175022×1252	1.6GB
add20 (vec)	MOS	5091	2479×42799	809.47MB
add20 (mat)	MOS	5091	18189×42799	5.8GB
smult20 (vec)	MOS	46075	28759×9048	1.94GB
smult20 (mat)	MOS	46075	213185×9048	14.37GB

A. Experimental Setup

Platform and software. We implement our proposed compression algorithm in C++ and integrate it into the adjoint sensitivity simulation. The platform used is Xyce [28], an open-source, SPICE-compatible circuit simulator from Sandia National Laboratories. The experiments are conducted on an AMD Ryzen 7 4800H CPU operating at a clock speed of 2.9 GHz.

Datasets. We evaluate the proposed error-bounded lossy compression algorithm on six datasets, consisting of vector and matrix data generated from simulations of both linear and

TABLE III: Comparison of compression ratio, compression time (in seconds), and decompression time (in seconds) for different compression methods.

Dataset	ZSTD [27]			MASC [8]			ZFP [19]			SZ [17]			Our work		
	CR	T_{comp}	T_{decomp}	CR	T_{comp}	T_{decomp}	CR	T_{comp}	T_{decomp}	CR	T_{comp}	T_{decomp}	CR	T_{comp}	T_{decomp}
ibm1t (vec)	1.45	4.49	1.85	1.98	5.23	2.72	4.08	1.3	1.38	11.6	2.02	1.4	12.78	1.97	1.34
ibm1t (mat)	29.59	1.73	1.67	7.17	11.35	8.61	3.54	3.62	0.65	196.23	5.45	2.24	970.11	5.29	1.85
add20 (vec)	6.03	3.63	2.09	2.95	6.79	3.97	5.02	1.79	1.77	76.08	2.67	1.33	72.28	2.76	1.45
add20 (mat)	9.74	10.68	4.92	11.90	36.72	29.28	4.48	12.6	12.72	80.46	18.6	8.52	125.85	18.01	5.52
smult20 (vec)	5.69	4.85	3.51	4.18	10.65	7.84	8.19	4.16	3.25	92.56	7.89	6.56	146.39	4.18	3.27
smult20 (mat)	12.18	25.23	15.72	11.07	86.51	75.06	16.97	29.82	18.68	169.25	50.28	25.72	205.63	45.42	18.17
Average	10.78	8.44	4.96	6.54	26.21	21.25	7.05	8.88	6.41	104.36	14.49	8.31	255.51	12.94	7.93

nonlinear circuits. As shown in Table II, the first column represents the name of the circuit and whether it is vector data or matrix data, the second column is the type of the circuit, the third column indicates the number of elements in the circuit, and the fourth and fifth columns respectively represent the dimensions and size of the data.

Baselines. We compare the proposed algorithm with the state-of-the-art lossless compressor ZSTD [27], the lossless compression algorithm MASC [8] specifically designed for matrices, and two cutting-edge lossy compressors, ZFP [19] and SZ [17]. All algorithms use the latest versions.

Configuration. In scientific computing, the absolute and relative error bounds for lossy compression are typically set to 10^{-5} and 10^{-3} [21], respectively, providing sufficient accuracy while significantly reducing simulation memory overhead. We adopt this configuration for our experiment as well.

B. Compression Performance

As shown in Table III, we compare the proposed error-bounded lossy compression algorithm with the aforementioned compression methods. In terms of compression ratio, our algorithm achieves average ratios of $23.70\times$ and $39.07\times$ higher than the state-of-the-art lossless compression methods ZSTD and MASC, respectively. This result highlights the superior capability of lossy compression in handling floating-point data. When compared to advanced lossy compression methods such as ZFP and SZ, the proposed algorithm achieves compression ratios of $36.24\times$ and $2.44\times$ higher, respectively. Thus, the simulation memory overhead can be reduced by two orders of magnitude. This improvement can be attributed to the algorithm's effective exploitation of the characteristics of circuit simulation data. By leveraging efficient smoothing techniques and predictive modeling, the algorithm effectively eliminates data redundancy in vectors and matrices. In terms of compression and decompression time, the time consumed by the five compression algorithms for both compression and decompression is negligible compared to the overall time cost of the sensitivity simulation itself.

C. Error Impact Evaluation

As shown in Figure 8, we integrate the proposed error-bounded lossy compression algorithm into the adjoint sensitivity simulation to analyze the impact of floating-point errors introduced by lossy compression on the accuracy of sensitivity

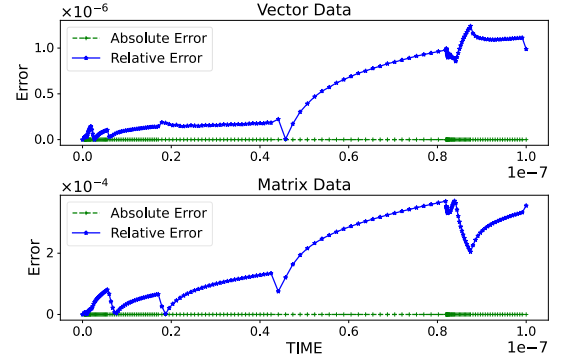


Fig. 8: Error comparison by integrating lossy compression algorithm into transient adjoint sensitive analysis with add20.

simulation results. The absolute error bound is set to 10^{-5} , and the relative error bound to 10^{-3} . Using the compression algorithm, we reduce the memory overhead caused by storing vectors and matrices during the simulation process. It can be observed that the sensitivity analysis results at each time point remained within the predefined error bounds after integrating the lossy compression. This is attributed to the proposed algorithm's strict quantization of errors introduced during the compression process. Therefore, the proposed error-bounded lossy compression significantly reduces memory overhead while effectively preserving simulation accuracy.

VII. CONCLUSION

This work introduces the first error-bounded lossy compression algorithm capable of efficiently compressing both vector and matrix data in circuit simulations. This approach significantly reduces memory overhead in adjoint sensitivity simulations. Based on the characteristics of data in circuit simulations, we propose an effective smoothing method tailored for simulation data. Subsequently, we design different data prediction algorithms for regions with distinct properties. Finally, through rigorous error quantization and efficient data encoding, the proposed approach significantly reduces the data size while preserving accuracy.

ACKNOWLEDGMENTS

The corresponding author is Zhou Jin. This work was supported by the NSFC (Grant Nos. 92473107, 62204265, 62234010, U23A20301, 62032023, and T2125013), and the Innovation Funding of ICT, CAS (Grant No. E461050).

REFERENCES

- [1] I. Stevanovic and C. McAndrew. Quadratic backward propagation of variance for nonlinear statistical circuit modeling. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, 2009.
- [2] B. Agrawal, F. Liu, and S. Nassif. Circuit optimization using scale based sensitivities. In *IEEE Custom Integrated Circuits Conference (CICC'06)*, 2006.
- [3] J. Sun, X. Zha, C. Wang, X. Wu, D. Niu, W. Xing, and Z. Jin. Pseudo adjoint optimization: Harnessing the solution curve for spice acceleration. In *ACM/IEEE International Conference on Computer-Aided Design (ICCAD'24)*, 2024.
- [4] C. Gu and J. Roychowdhury. An efficient, fully nonlinear, variability-aware non-monte-carlo yield estimation procedure with applications to sram cells and ring oscillators. In *Asia and South Pacific Design Automation Conference (ASPDAC'08)*, 2008.
- [5] K. V. Aadithya, E. Keiter, and T. Mei. Dagsens: Directed acyclic graph based direct and adjoint transient sensitivity analysis for event-driven objective functions. In *IEEE/ACM International Conference on Computer-Aided Design (ICCAD'17)*, 2017.
- [6] Y. Cao, S. Li, L. R. Petzold, and R. Serban. Adjoint sensitivity analysis for differential-algebraic equations: The adjoint dae system and its numerical solution. *SIAM Journal on Scientific Computing*, 2002.
- [7] F. Liu and P. Feldmann. A time-unrolling method to compute sensitivity of dynamic systems. In *ACM/IEEE Design Automation Conference (DAC'14)*, 2014.
- [8] C. Li, B. Zhang, Y. Duan, Y. Li, Z. Ye, W. Liu, D. Tao, and Z. Jin. Masc: A memory-efficient adjoint sensitivity analysis through compression using novel spatiotemporal prediction. In *ACM/IEEE Design Automation Conference (DAC'24)*, 2024.
- [9] J. Roychowdhury. *Numerical Simulation and Modelling of Electronic and Biochemical Systems*, 2009.
- [10] R. Li, Z. Li, Y. Wu, C. Chen, and Y. Zheng. Elf: Erasing-based lossless floating-point compression. In *VLDB Endowment*, 2023.
- [11] X.-C. Wu, S. Di, E. M. Dasgupta, F. Cappello, H. Finkel, Y. Alexeev, and F. T. Chong. Full-state quantum circuit simulation by using data compression. In *international Conference for High Performance Computing, Networking, Storage and Analysis (SC'19)*, 2019.
- [12] L. Li and W. Yu. Efficient and accuracy-ensured waveform compression for transient circuit simulation. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, 2020.
- [13] L. Li, W. Yu, G. Guo, and Z. Zhou. More efficient accuracy-ensured waveform compression for circuit simulation supporting asynchronous waveforms. In *Great Lakes Symposium on VLSI (GLSVLSI'23)*, 2023.
- [14] W. Hu, Z. Ye, and Y. Wang. Adjoint transient sensitivity analysis for objective functions associated to many time points. In *ACM/IEEE Design Automation Conference (DAC'20)*, 2020.
- [15] N. Sagan and J. Roychowdhury. Transient adjoint dae sensitivities: A complete, rigorous, and numerically accurate formulation. In *Asia and South Pacific Design Automation Conference (ASPDAC'22)*, 2022.
- [16] IEEE Standards Committee et al. 754-2008 ieee standard for floating-point arithmetic, 2008.
- [17] X. Liang, S. Di, D. Tao, S. Li, S. Li, H. Guo, Z. Chen, and F. Cappello. Error-controlled lossy compression optimized for high compression ratios of scientific datasets. In *IEEE International Conference on Big Data (Big Data'18)*, 2018.
- [18] P. Lindstrom and M. Isenburt. Fast and efficient compression of floating-point data. *IEEE Transactions on Visualization and Computer Graphics*, 2006.
- [19] P. Lindstrom. Fixed-rate compressed floating-point arrays. *IEEE Transactions on Visualization and Computer Graphics*, 2014.
- [20] A. Eldstål-Ahrens and I. Sourdis. Memsz: Squeezing memory traffic with lossy compression. *ACM Transactions on Architecture and Code Optimization (TACO'20)*, 2020.
- [21] S. Jin, C. Zhang, X. Jiang, Y. Feng, H. Guan, G. Li, S. L. Song, and D. Tao. Comet: a novel memory-efficient deep learning training framework by using error-bounded lossy compression. In *VLDB Endowment*, 2021.
- [22] Y. Huang, K. Zhao, S. Di, G. Li, M. Dmitriev, T.-L. D. Tonellot, and F. Cappello. Towards improving reverse time migration performance by high-speed lossy compression. In *IEEE/ACM International Symposium on Cluster, Cloud and Internet Computing (CCGrid'23)*. IEEE, 2023.
- [23] S. Lakshminarasimhan, N. Shah, S. Ethier, S.-H. Ku, C.-S. Chang, S. Klasky, R. Latham, R. Ross, and N. F. Samatova. Isabela for effective in situ compression of scientific data. *Concurrency and Computation: Practice and Experience*, 2013.
- [24] J. Huang, J. Liu, S. Di, Y. Zhai, Z. Jian, S. Wu, K. Zhao, Z. Chen, Y. Guo, and F. Cappello. Exploring wavelet transform usages for error-bounded scientific data compression. In *IEEE International Conference on Big Data (BigData'23)*, 2023.
- [25] Y. Su, F. Yang, and X. Zeng. Amor: An efficient aggregating based model order reduction method for many-terminal interconnect circuits. In *Design Automation Conference (DAC'12)*, 2012.
- [26] P. Liakos, K. Papakonstantinou, and Y. Kotidis. Chimp: Efficient lossless floating point compression for time series databases. In *VLDB Endowment*, 2022.
- [27] J. Chen, M. Daverveldt, and Z. Al-Ars. Fpga acceleration of zstd compression algorithm. In *IEEE International Parallel and Distributed Processing Symposium Workshops (IPDPSW'21)*, 2021.
- [28] J. Verley, E. Keiter, and H. Thornquist. Xyce: Open source simulation for large-scale circuits. Technical report, Sandia National Lab. (SNL-NM), 2018.